Enterprise Integration Patterns — Entra ID ↔ Google + M365 Coexistence + Third-Party Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Enterprise Integration Patterns (Entra ID ↔ Google + M365 Coexistence + Third-Party Apps) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Enterprise Integration Patterns (Entra ID ↔ Google + M365 Coexistence + Third-Party Apps) 5

Why a pattern catalog, not a pile of connectors . . . 5

The five integration dimensions . . . 5

Reference patterns in this catalog . . . 6

Anti-patterns the catalog exists to kill . . . 6

What you can control . . . 6

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Enterprise Integration Patterns 7

2.1 Both control planes accessible (Entra + Google) . . . 8

2.2 Administrative roles (least privilege) . . . 8

2.3 Chosen identity provider and direction, per workload . . . 8

2.4 Single source of truth for identity + provisioning model . . . 8

2.5 Third-party app SSO/provisioning capability matrix . . . 9

2.6 Coexistence cutover plan (M365 ↔ Google) . . . 9

2.7 API / data-access identities defined . . . 9

2.8 Microsoft Graph PowerShell SDK . . . 10

2.9 Pilot app set and change control . . . 10

2.10 Break-glass accounts . . . 10

Checklist . . . 11

Global Requirements — Google Workspace & Gemini Solutions 11

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 12

3. DNS control for the routed domains . . . 12

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 13

3. Implementation Guide — The Reference Integration Patterns 13
Step 0 — Inventory and classify (do this first) . . . 13
Section A — Entra ID ↔ Google Workspace . . . 14
Section B — M365 ↔ Google coexistence . . . 15
Section C — Third-party enterprise apps . . . 15
Section D — API & data access (cross-cutting) . . . 16
Step N — Pilot, validate, expand . . . 17
5. Testing Guide — Pattern-Conformance Gates 17
Test 1 — Estate inventory is complete . . . 18
Test 2 — Every integration maps to a pattern (no snowflakes) . . . 18
Test 3 — Entra↔Google federation conforms (P1) . . . 18
Test 4 — Provisioning has one source of truth (P2/P3) . . . 18
Test 5 — Third-party app SSO is federated (P5) . . . 18
Test 6 — Third-party app provisioning + de-provisioning (P6) . . . 18
Test 7 — API/data access uses scoped, short-lived credentials (P7) . . . 18
Test 8 — Coexistence cutover is defined (P4) . . . 18
Test 9 — Remediation preview is accurate . . . 19
Pass/fail gate . . . 19
6. Troubleshooting Guide — Enterprise Integration Patterns 19
A. Discovery or report returns nothing / errors . . . 19
B. Entra↔Google SSO fails (P1) . . . 20
C. Provisioning drift or duplicates (P2/P3) . . . 20
D. Third-party app SSO/provisioning (P5/P6) . . . 20
E. API / data access (P7) . . . 21
F. Coverage/exception report shows unclassified integrations . . . 21
G. A legitimate integration broke after remediation . . . 21
H. Diagnostics quick reference . . . 21
I. When to escalate . . . 22
7. Best Practices — Enterprise Integration Patterns 22
8. Deliverables Summary — Enterprise Integration Patterns 23
Documentation . . . 23
PowerShell scripts . . . 23
Integration-Patterns reference doc — outline . . . 24
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Enterprise Integration Patterns (Entra ID ↔ Google + M365 Coexistence + Third-Party Apps)

White-label · Google Category

During a Google Workspace transformation, identity has to keep working everywhere at once — across **Microsoft Entra ID**, **Google Workspace / Cloud Identity**, the **Microsoft 365** systems still live in coexistence, and a long tail of **third-party enterprise apps** (ServiceNow, SAP, Databricks, and more). Left to each team, integrations become snowflakes: one-off SAML apps, hand-rolled syncs, shared service accounts, and passwords in scripts. The estate ends up inconsistent, over-permissioned, and impossible to reason about — and the connector count only grows.

This package answers that with a **pattern catalog**: a small, named set of reference integration designs — for identity federation, provisioning, app SSO (SAML/OIDC), API/data access, and coexistence — that every new integration is expected to conform to. Instead of deciding authentication and authorization app-by-app, an architect picks the matching pattern and applies it. Consistency becomes the default; exceptions become visible and few.

What you get

• A plain-language Solution Overview of the pattern-catalog approach — the five integration

dimensions, the seven reference patterns (P1–P7), and the anti-patterns (per-app snowflakes, credential sprawl, two sources of truth) the catalog exists to eliminate.

• A Requirements Guide with verify / implement-if-missing for every prerequisite.

• An Implementation Guide that is the catalog itself: **Entra↔Google** federation and provisioning

(SAML SSO, SCIM push, GCDS pull), **M365 coexistence** with a defined cutover, **third-party apps** (ServiceNow, SAP, Databricks) with the recommended authN/authZ and data-access approach for each, and a cross-cutting **API/data-access** pattern — plus the standards and anti-patterns.

• Five PowerShell scripts for estate discovery, pattern-conformance validation, an integration

coverage/exception report, safe remediation of snowflakes, and a pilot-scope builder.

• A Testing Guide of conformance gates and a Troubleshooting Guide for a confident, staged rollout,

plus the outline for a maintained Integration-Patterns reference doc.

What this delivers

A consistent, standards-based integration estate — federated authentication, group-driven authorization, one source of truth per object, and scoped, short-lived credentials for API and data access — with the third-party-app internals correctly deferred to the vendors' own docs and integrated only at the pattern level. Snowflakes and credential sprawl are designed out; every integration maps to a named pattern, and the few deliberate exceptions are documented and owned.

The standard deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

One consistent way to connect Google, Microsoft, and everything else — stable identity and data access, no snowflake connections — that scales through the transformation and beyond, proven on a pilot before you touch the whole estate.

1. Solution Overview — Enterprise Integration Patterns (Entra ID ↔ Google + M365 Coexistence + Third-Party Apps)

Why a pattern catalog, not a pile of connectors

During a Google Workspace transformation, identity does not sit still. Users, groups, apps, and data have to keep working across **Microsoft Entra ID**, **Google Workspace / Cloud Identity**, the **Microsoft 365** systems still live during coexistence, and a long tail of **third-party enterprise apps** (ServiceNow, SAP, Databricks, and more). The failure mode is predictable: every team that needs an integration builds its own — a one-off SAML app here, a hand-rolled sync there, a service account with a permanent password somewhere else. The result is a **snowflake estate**: inconsistent authentication, ad-hoc authorization, credential sprawl, and coexistence gaps that nobody owns.

This solution answers that with a **pattern catalog**: a small, named set of reference integration designs that every new integration is expected to conform to. Instead of deciding authentication and authorization app-by-app, an architect picks the matching pattern and applies it. Consistency becomes the default, exceptions become visible and few, and the estate scales through the transformation and beyond.

> *This is an **architecture / governance** product. It defines the reference patterns, the standards, and the anti-patterns, and ships scripts that inventory the estate and check conformance. It is not a click-by-click configuration guide for any single app — third-party enterprise-app internals are integrated **at the pattern level** and their console steps are deferred to the vendor's own docs.*

The five integration dimensions

Every integration in the estate is described by where it sits on five dimensions. The catalog names a recommended pattern for each.

1. **Identity federation** — who is the identity provider (IdP) and who is the relying party (SP/RP),

and over what protocol (SAML 2.0 or OIDC). During a Google transformation, Entra ID is commonly the enterprise IdP and Google Workspace a relying party, but the direction can invert per workload.

1. **Provisioning** — how accounts and groups are created, updated, and de-provisioned across

directories: **SCIM** push (Entra → Google via the gallery app), **GCDS** pull (AD/LDAP → Google), and **just-in-time (JIT)** creation on first SSO. Choose one authoritative source per object.

1. **App SSO (SAML / OIDC)** — how enterprise apps authenticate users. The pattern names the IdP,

the protocol, and how roles/entitlements flow (claims/groups).

1. **API & data access** — how services and automations authenticate to each other and to data:

OAuth 2.0 client-credentials / workload identity federation, service accounts, and least-privilege scoping — never long-lived shared secrets by default.

1. **Coexistence** — how identity and access stay consistent while M365 and Google run side by side: one source of truth, synchronized identities, and a defined cutover order per workload.

Reference patterns in this catalog

Each is named once and reused throughout the guides:

- **P1 — Entra-as-IdP → Google (SAML SSO)**: Entra ID federates sign-in to Google Workspace / Cloud Identity via the "Google Cloud / Google Workspace" gallery application.
- **P2 — Entra → Google provisioning (SCIM push)**: Entra ID provisions and de-provisions users and groups into Google via SCIM using the same gallery app.
- **P3 — AD-source provisioning (GCDS pull)**: where on-prem Active Directory / LDAP remains the source of truth, Google Cloud Directory Sync pulls objects into Google.
- **P4 — Coexistence identity (single source of truth)**: one authoritative directory, synchronized to the other ecosystem, with a per-workload cutover so authentication stays consistent during overlap.
- **P5 — Third-party app SSO (federated SAML/OIDC)**: enterprise apps federate to the chosen IdP over SAML or OIDC, with roles carried in claims/groups — no local passwords.
- **P6 — Third-party app provisioning (SCIM / connector)**: lifecycle into the app via SCIM where the app and IdP support it, or the app's supported connector where they do not.
- **P7 — API / data access (OAuth workload identity)**: service-to-service and data access via OAuth client-credentials / workload identity federation and scoped service accounts, not shared secrets.

Anti-patterns the catalog exists to kill

- **Per-app snowflakes** — every app authenticated a different way, with no reuse.
- **Credential sprawl** — shared service accounts, long-lived secrets, and passwords in scripts.
- **Two sources of truth** — an object mastered in both directories, so they drift and conflict.
- **Over-broad authorization** — app roles and API scopes granted far beyond what the integration uses, expanding blast radius.
- **Undocumented coexistence** — no defined cutover order, so mail, SSO, or provisioning breaks mid-transition.

What you can control

- **Inventory** — every federation trust, SSO app, provisioning connection, and service credential across Entra, Google, and the third-party apps in scope.

• **Conformance** — whether each integration matches a catalog pattern, and where the exceptions are.

• **Standards** — the consistent authN (federated SSO), authZ (claims/group-driven, least privilege), and data-access (scoped OAuth / workload identity) approach applied everywhere.

• **Coexistence** — one source of truth, synchronized, with a documented cutover per workload.

Benefits

• **Consistency** — one way to connect Google, Microsoft, and everything else; new integrations reuse a pattern instead of inventing one.

• **Security** — federated SSO removes local passwords, least-privilege scoping bounds blast radius, and credential sprawl is designed out.

• **Visibility** — a real inventory and a conformance/exception view, so snowflakes are caught early.

• **Scale** — the estate grows through the transformation without the connector count becoming chaos.

Limitations (be honest with buyers)

• This is a **pattern and governance** product. It standardizes and inventories; it does not replace each vendor's own configuration documentation for the app internals.

• **Third-party app capabilities vary and change.** Whether a given app supports SCIM from a given IdP, or only SAML, or only a legacy connector, must be **verified in the vendor's current docs and your tenant** — the catalog names the pattern; the app decides what it supports.

• Some Entra provisioning and conditional-access capabilities require **Entra ID P1/P2** licensing (see Requirements).

• The scripts inventory and check conformance on the **Microsoft/Entra side** via Microsoft Graph (read-only); Google-side and third-party-side state is confirmed in those consoles.

The standard deployment model

> *Inventory the integration estate → classify each integration against the catalog patterns → define the target patterns and standards → pilot the target patterns on a scoped set of apps → validate conformance → expand across the estate, tracking exceptions. Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Enterprise Integration Patterns

Verify every requirement before defining or piloting the target patterns. Each entry: **Description / Why / Verify / Implement-if-missing.** Category-wide prerequisites (Google tenant, source M365/Entra environment, DNS control, source of truth, pilot scope, governance stakeholders) are in `global-requirements-google.md` — this guide adds what the integration-pattern work needs on top

of those.

2.1 Both control planes accessible (Entra + Google)

- **Description:** Administrative access to **Microsoft Entra ID** (to author federation and provisioning on the Microsoft side) and to the **Google Admin console / Cloud Identity** (to accept the SAML profile and enable SCIM on the Google side).
- **Why required:** The Entra↔Google patterns (P1/P2) are configured on both ends — Entra publishes SSO/SCIM; Google accepts the SSO profile and enables the SCIM endpoint.
- **Verify:** Sign in to **entra.microsoft.com** with an app-management role and to **admin.google.com** as a super-admin.
- **Implement:** Obtain the roles from the respective tenant owners before piloting.

2.2 Administrative roles (least privilege)

- **Description:** On Entra: `Application Administrator` (or `Cloud Application Administrator`) for federation/provisioning app configuration; `Global Administrator` only for tenant-wide settings. On Google: a **super-admin** for the SSO profile and provisioning; on third-party apps, the app's SSO/admin role.
- **Why required:** Configuring federation, SSO, and provisioning are privileged actions on each plane.
- **Verify:** Entra → `Roles and administrators`; Google → `Account → Admin roles`; each app's admin console.
- **Implement:** Assign the least-privilege role on each plane; reserve `Global Administrator` for the one-time tenant settings.

2.3 Chosen identity provider and direction, per workload

- **Description:** A documented decision on **which system is the IdP** for each workload and app, and the **protocol** (SAML 2.0 or OIDC). During coexistence the answer can differ per workload.
- **Why required:** The catalog patterns are direction-specific (P1 = Entra-as-IdP → Google). Picking the IdP per workload up front prevents circular or conflicting trusts.
- **Verify:** A short IdP-direction table exists for the workloads/apps in scope.
- **Implement:** Decide the enterprise IdP (commonly Entra ID during a Google transformation) and any per-workload exceptions before configuring; record it in the Integration-Patterns reference doc.

2.4 Single source of truth for identity + provisioning model

- **Description:** One authoritative directory per object type and a chosen provisioning mechanism —

SCIM push (Entra → Google, P2), **GCDS pull** (AD/LDAP → Google, P3), or **JIT** on first SSO — never two masters for the same object.

• **Why required:** Two sources of truth is the anti-pattern that causes directories to drift and conflict; consistent provisioning underpins SSO, licensing, and de-provisioning.

• **Verify:** Confirm which directory masters users/groups and which mechanism moves them into Google (and into each third-party app).

• **Implement:** Pick the source of truth and the P2/P3/JIT mechanism per object before enabling provisioning. (Enterprise-source-of-truth details are in global requirement 4.)

2.5 Third-party app SSO/provisioning capability matrix

• **Description:** For each in-scope third-party app (e.g. ServiceNow, SAP, Databricks), the **verified** facts: does it federate over SAML, OIDC, or both; does it support **SCIM** provisioning from the chosen IdP, or only a legacy/connector approach; how are roles carried (claims/groups).

• **Why required:** App capabilities differ and change — e.g. Databricks supports SAML/OIDC + SCIM (with JIT on newer accounts), while some apps support SAML SSO but **not** SCIM from a given IdP and need a supported connector instead. The pattern chosen (P5/P6) depends on what the app actually supports.

• **Verify:** Confirm each app's current capability from the **vendor's own documentation** and a test in your tenant — do not assume.

• **Implement:** Build the matrix (app → protocol → provisioning → role model) and record it in the Integration-Patterns reference doc; flag any "verify in tenant" items.

2.6 Coexistence cutover plan (M365 ↔ Google)

• **Description:** A per-workload plan for the coexistence window: which IdP owns each workload, how identities stay synchronized, and the **cutover order** (identity → SSO → provisioning → mail/data).

• **Why required:** Undocumented coexistence is an anti-pattern; a defined order keeps authentication and provisioning consistent while both ecosystems are live.

• **Verify:** A cutover plan exists with an owner and change windows.

• **Implement:** Draft the plan under change control before piloting coexistence patterns (mail/DNS changes are high-impact — see global requirement 3).

2.7 API / data-access identities defined

• **Description:** For service-to-service and data access (P7): the intended **OAuth client-credentials / workload identity federation **and** scoped service accounts**, with least-privilege scopes — no shared long-lived secrets by default.

• **Why required:** Credential sprawl and over-broad scopes are the API-layer anti-patterns; workload identity federation removes standing secrets.

• **Verify:** Each automation/data path has a named identity and a scoped set of permissions.

• **Implement:** Define the workload-identity or service-account model and scopes before granting access.

2.8 Microsoft Graph PowerShell SDK

• **Description:** The Microsoft Graph PowerShell SDK on the operator workstation, with consent to the read scopes the scripts use.

• **Why required:** Discovery, validation, and reporting read the Entra-side integration estate — federation/SSO apps, provisioning jobs, service principals, app-role assignments, OAuth grants, and credentials — through Microsoft Graph.

• **Verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; a test `Connect-MgGraph` succeeds.

• **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`; the scripts request least privilege (`Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`, `Synchronization.Read.All`; and — only for remediation — `Application.ReadWrite.All` / `DelegatedPermissionGrant.ReadWrite.All`).

2.9 Pilot app set and change control

• **Description:** A small, representative **set of apps** to pilot the target patterns on (typically: the Entra↔Google trust plus one third-party app), and a change-control window.

• **Why required:** Every pattern is validated on a pilot before the estate; a scoped set proves the design without risking production integrations.

• **Verify:** A named pilot app set and change window exist.

• **Implement:** Choose the pilot apps and agree the change process; use `GWINTEG-Set-ScopeTargeting.ps1` to build the pilot scope group.

2.10 Break-glass accounts

• **Description:** Two cloud-only emergency-access accounts excluded from federation/conditional-access policies.

• **Why required:** A federation or SSO change could interfere with admin sign-in; break-glass accounts

restore access.

- **Verify:** Entra → `Users`; two cloud-only `@onmicrosoft.com` admins exist, excluded from broad policies.
- **Implement:** Create them, exclude from federation/CA policies, store credentials securely, monitor sign-ins.

Checklist

- Both control planes accessible — Entra (app-mgmt role) + Google (super-admin)
- Least-privilege admin roles assigned on each plane
- IdP + direction + protocol decided per workload/app
- Single source of truth + provisioning model (SCIM / GCDS / JIT) chosen per object
- Third-party app SSO/provisioning capability matrix built and verified from vendor docs
- Coexistence cutover plan drafted under change control
- API / data-access identities and scopes defined (workload identity / scoped service accounts)
- Microsoft Graph PowerShell SDK installed and consented
- Pilot app set and change window named — `GWINTEG-Set-ScopeTargeting.ps1`
- Two break-glass accounts created and excluded from federation/CA policies

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

- **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).
- **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — The Reference Integration Patterns

This is the catalog. Each pattern names its **intent**, **authN**, **authZ**, **data access**, the **standard** to hold to, and the **anti-pattern** to avoid. The pattern names (P1–P7) are used throughout the other guides and in the Integration-Patterns reference doc.

> *Model: inventory the estate → classify each integration against the catalog → pilot the target patterns on the scoped app set → validate conformance → expand, tracking exceptions. Every change is staged, validated on a pilot, and reversible. **Vendor boundary.** The **Entra** side is authored from Microsoft's public docs; the **Google Admin console / Cloud Identity** side from Google's public docs. **Third-party enterprise-app internals** (ServiceNow, SAP, Databricks, …) are integrated **at the pattern level** — the recommended authN/authZ/data-access approach — and their exact console steps and current capabilities are **deferred to the vendor's own documentation and verified in your tenant**.*

Step 0 — Inventory and classify (do this first)

1. Run `GWINTEG-Discovery.ps1` to inventory the **Entra-side** estate: SSO/federation apps,

provisioning (SCIM synchronization) jobs, service principals, app-role assignments, delegated OAuth grants, and credentials.

1. On **Google**, record the SSO profile(s) under `Security → Authentication → SSO with third-party IdP` and any provisioning/GCDS in use.

1. For each **third-party app** in scope, record its SSO protocol and provisioning mechanism into the capability matrix (Requirement 2.5) — verified from the vendor's docs.

1. Run `GWINTEG-Report.ps1` to produce the coverage/exception view: which integrations match a catalog pattern and which are snowflakes.

Section A — Entra ID ↔ Google Workspace

P1 — Entra-as-IdP → Google (SAML SSO)

• **Intent:** Entra ID federates user sign-in to Google Workspace / Cloud Identity so users authenticate once against the enterprise IdP.

• **AuthN:** SAML 2.0. In Entra, add the **"Google Cloud / Google Workspace"** gallery application and configure SAML SSO. In the **Google Admin console** → `Security → Authentication → SSO with third-party IdP → Third-party SSO profiles → Add SAML profile`, paste Entra's IdP metadata; Google presents the ACS URL and Entity ID that Entra must return.

• **AuthZ:** Google authorization (OU/group membership, admin roles) is decided in Google; the SAML assertion authenticates, it does not authorize Google admin. Keep the **NameID** aligned to the Google primary domain — if the Entra domain differs, map the NameID claim to a value that matches the verified Google domain.

• **Data access:** none directly — this is sign-in federation.

• **Standard:** the Google domain is **verified in Entra**; NameID matches the Google identity; SSO is piloted on an OU before org-wide enforcement.

• **Anti-pattern:** enabling SSO org-wide without a break-glass Google super-admin path, or a NameID that does not match the Google domain (users cannot sign in).

P2 — Entra → Google provisioning (SCIM push)

• **Intent:** Entra ID is the source of truth; it creates, updates, and de-provisions Google users and groups automatically.

• **AuthN/mechanism:** SCIM. In the same gallery app, configure **Provisioning** (SCIM). On Google, enable the SCIM/provisioning endpoint and authorize Entra (a Google admin identity for the connection). Assign users/groups to the app to bring them into scope; start provisioning.

• **AuthZ:** attribute mappings decide what Entra writes to Google (including OU placement where mapped); scope provisioning by assignment or scoping filter so only intended users flow.

• **Data access:** directory objects only, over SCIM.

• **Standard:** one source of truth (Entra) for the objects it masters; **SCIM push (P2)** and **GCDS pull (P3)** are not both used for the same object.

• **Anti-pattern:** SCIM push **and** GCDS pull mastering the same users — they fight and drift.

P3 — AD-source provisioning (GCDS pull)

• **Intent:** where **on-prem Active Directory / LDAP remains the source of truth**, Google is populated by pulling from that directory.

• **Mechanism: Google Cloud Directory Sync (GCDS)** reads AD/LDAP and syncs users/groups into Google on a schedule.

• **AuthZ:** GCDS mapping rules decide which OUs/attributes flow.

• **Standard:** use P3 when AD/LDAP is authoritative; use P2 when **Entra** is authoritative. Pick one per object, never both.

• **Anti-pattern:** running GCDS against objects that Entra also pushes via SCIM (two masters).

JIT note: SAML **just-in-time** creation makes a Google user on first SSO. Treat JIT as a convenience, not the source of truth — for lifecycle (especially **de-provisioning**), use P2 or P3 so leavers are actually removed.

Section B — M365 ↔ Google coexistence

P4 — Coexistence identity (single source of truth)

• **Intent:** keep authentication and provisioning consistent while M365 and Google run side by side.

• **AuthN:** one enterprise IdP owns sign-in for each workload (commonly Entra ID during a Google transformation); relying parties on both sides federate to it (P1/P5).

• **Provisioning:** one authoritative directory, synchronized into the other ecosystem (P2/P3), so an object exists once and flows out — never mastered in both.

• **Data access:** governed by each ecosystem's own controls; the coexistence layer keeps **identity** consistent, not data merged.

• **Standard:** a documented **cutover order** per workload — identity → SSO → provisioning → mail/data — under change control.

• **Anti-pattern:** undocumented coexistence — no defined IdP owner or cutover order — so SSO, provisioning, or mail flow breaks mid-transition.

Section C — Third-party enterprise apps

> ***The pattern is the deliverable here.*** *For each app, the recommended authN/authZ/data-access approach below is the guidance; the* ***exact console steps and current capabilities are the vendor's and must be verified*** *(Requirement 2.5). Capabilities change — treat any specific claim as "verify in the vendor's current docs and your tenant."*

P5 — Third-party app SSO (federated SAML/OIDC)

- **Intent:** enterprise apps authenticate users against the chosen IdP — no local app passwords.
- **AuthN:** SAML 2.0 or OIDC to the IdP. Prefer the app's **gallery/marketplace integration** on the IdP where one exists.
- **AuthZ:** carry roles/entitlements in the assertion or token — **group or claim-driven** — so app authorization follows directory groups, not per-user local admin.
- **Standard:** federated SSO for every app that supports it; roles from groups/claims; least-privilege app roles.
- **Anti-pattern:** local app accounts with their own passwords (a snowflake and a credential-sprawl risk).

P6 — Third-party app provisioning (SCIM / connector)

- **Intent:** create, update, and de-provision app accounts automatically from the source of truth.
- **Mechanism: SCIM** from the chosen IdP where **both the IdP and the app support it**; otherwise the app's **supported connector** or import (e.g. a SOAP/API-based provisioning integration).
- **AuthZ:** map groups → app roles so entitlements follow directory membership; de-provision on leave.
- **Standard:** automated lifecycle with de-provisioning; group-to-role mapping; scoped provisioning.
- **Anti-pattern:** manual account creation and, worse, **no de-provisioning** — leavers keep access.

Per-app pattern application (verify each in vendor docs + your tenant)

- **ServiceNow** — **P5:** SAML 2.0 SSO to the IdP (multi-provider SSO supported). **P6:** provisioning approach must be **verified** — SCIM support from a given IdP has changed over time and some configurations require the app's supported (e.g. SOAP/API) provisioning instead; do not assume SCIM. Roles map from groups → ServiceNow roles.
- **SAP** — **P5:** SAML 2.0 / OIDC federation, commonly fronted by the customer's SAP identity/BTP identity service brokering to the enterprise IdP; **verify the exact brokering topology** for the SAP products in scope. **P6:** provisioning via the app's supported mechanism. **P7:** API/data access via OAuth to SAP APIs with scoped clients.
- **Databricks** — **P5:** SAML 2.0 **or** OIDC SSO to the IdP (both supported). **P6: SCIM** provisioning from the IdP; **JIT** on first SSO is on by default for newer accounts — **verify** the behavior for your workspace/account age. Roles/groups map to Databricks entitlements.

Section D — API & data access (cross-cutting)

P7 — API / data access (OAuth workload identity)

• **Intent:** services, automations, and data pipelines authenticate to each other and to data without standing shared secrets.

• **AuthN: OAuth 2.0 client-credentials** or **workload identity federation** (an external workload presents a trusted token and exchanges it for short-lived access — no stored secret); **scoped service accounts** where a service identity is required.

• **AuthZ:** least-privilege scopes / roles for exactly the APIs and data the integration uses.

• **Data access:** through the granted scopes only; prefer read-only where the integration only reads.

• **Standard:** short-lived, federated credentials; least-privilege scopes; a named owner per identity.

• **Anti-pattern: credential sprawl** — shared service accounts, long-lived secrets, passwords in scripts, or `*.ReadWrite.All`-style scopes granted "to be safe."

Step N — Pilot, validate, expand

1. **Pilot** the target patterns on the scoped app set (Requirement 2.9): the Entra↔Google trust (P1/P2 or P3) plus one third-party app (P5/P6) and one data path (P7).

1. **Validate** conformance with `GWINTEG-Validation.ps1` and the Testing Guide gates.

2. **Remediate** snowflakes with `GWINTEG-Remediation.ps1` (preview `-WhatIf` first, confirm with the app owner, then apply to the reviewed integration only).

1. **Expand** across the estate in batches, re-running `GWINTEG-Discovery.ps1` and `GWINTEG-Report.ps1` on a cadence so new integrations are classified as they appear; record every deliberate exception.

Order of operations

```
Inventory + classify → target patterns + standards → pilot (Entra↔Google +
one app + one data path) → validate conformance → remediate snowflakes →
expand, tracking exceptions
```

Rollback

• **P1 SSO:** disable the Google third-party SSO profile / disable the Entra SSO app — sign-in reverts to Google-native or the prior IdP. Keep a break-glass Google super-admin outside SSO.

• **P2 SCIM:** stop provisioning in the Entra app — existing objects remain; no new writes.

• **P5/P6 app:** disable the app's federation/provisioning to revert to its prior auth; every pattern change is piloted and reversible before estate-wide expansion.

5. Testing Guide — Pattern-Conformance Gates

Run after piloting the target patterns (Implementation Step N) and on the recurring governance cadence. Each test checks whether the estate conforms to the catalog — not just that a single app works, but that it works **the standard way**.

Test 1 — Estate inventory is complete

Do: Run `GWINTEG-Discovery.ps1`. **Expected:** Every Entra-side SSO/federation app, provisioning (SCIM) job, service principal, app-role assignment, delegated grant, and credential is listed — a usable baseline. Google-side SSO profiles and third-party apps are recorded in the capability matrix.

Test 2 — Every integration maps to a pattern (no snowflakes)

Do: Run `GWINTEG-Report.ps1` (coverage/exception section). **Expected:** Each integration is classified against a catalog pattern (P1–P7); the exception list contains only **deliberate, documented** exceptions — unclassified snowflakes are flagged.

Test 3 — Entra↔Google federation conforms (P1)

Do: Run `GWINTEG-Validation.ps1`; a pilot-OU user signs in to Google. **Expected:** SSO is via Entra (SAML), the NameID matches the Google domain, and the user authenticates. A break-glass Google super-admin can still sign in outside SSO.

Test 4 — Provisioning has one source of truth (P2/P3)

Do: Run `GWINTEG-Validation.ps1` (provisioning section). **Expected:** Each provisioned object type has exactly **one** master — SCIM push (P2) **or** GCDS pull (P3), never both; de-provisioning is in place (a disabled/removed source user is de-provisioned, not orphaned).

Test 5 — Third-party app SSO is federated (P5)

Do: For a pilot third-party app, a pilot user signs in via the IdP. **Expected:** The user authenticates through the chosen IdP over SAML/OIDC with **no local app password**, and app roles are driven by group/claim — not per-user local admin.

Test 6 — Third-party app provisioning + de-provisioning (P6)

Do: Add and then remove a test user from the source group for a pilot app. **Expected:** The user is provisioned into the app on add and **de-provisioned on remove**, via the app's verified mechanism (SCIM or supported connector). No manual account creation.

Test 7 — API/data access uses scoped, short-lived credentials (P7)

Do: Run `GWINTEG-Report.ps1` (credential/scope section) for the pilot data path. **Expected:** Service-to-service and data access use OAuth client-credentials / workload identity federation with **least-privilege scopes**; no long-lived shared secrets or over-broad (`*.ReadWrite.All`) scopes on the integration.

Test 8 — Coexistence cutover is defined (P4)

Do: Review the coexistence cutover plan against the live state. **Expected:** Each workload has a named IdP owner and a documented cutover order; identity stays consistent across M365 and Google during overlap.

Test 9 — Remediation preview is accurate

Do: Run `GWINTEG-Remediation.ps1` (default `-WhatIf`) against a flagged snowflake, then re-run with `-WhatIf:$false`. **Expected:** The preview lists exactly the change it will make; after applying, `GWINTEG-Validation.ps1` confirms the integration now conforms.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 pass on the pilot app set; one third-party app conforms (Tests 5–6); the pilot data path conforms (Test 7)
Expand pass	Tests 1–4, 7–8 pass across the estate; the exception list is reviewed and every item is deliberate; discovery + report run on a schedule

6. Troubleshooting Guide — Enterprise Integration Patterns

Use `GWINTEG-Discovery.ps1` for the Entra-side estate, `GWINTEG-Report.ps1` for coverage/exceptions and credential/scope risk, `GWINTEG-Validation.ps1` for pattern conformance, and `GWINTEG-Remediation.ps1` (preview first) for safe fixes. Google-side and third-party-app state is confirmed in those consoles.

A. Discovery or report returns nothing / errors

Symptom	Cause	Remediation
`Connect-MgGraph` fails or scopes missing	SDK not installed, or read scopes not consented	`Install-Module Microsoft.Graph -Scope CurrentUser`; re-run and accept `Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`, `Synchronization.Read.All`
Empty results	Signed in to the wrong tenant, or an app-read role missing	Confirm the tenant; ensure the operator holds `Application Administrator` / `Cloud Application Administrator`
Provisioning jobs not listed	`Synchronization.Read.All` not granted	Re-run and consent the synchronization read scope

B. Entra↔Google SSO fails (P1)

Symptom	Cause	Remediation
Users cannot sign in to Google after enabling SSO	NameID does not match the Google primary domain	In the Entra "Google Cloud / Google Workspace" app, map the NameID claim to a value matching the verified Google domain
SSO loop or "invalid assertion"	ACS URL / Entity ID mismatch between Entra and the Google SAML profile	Re-copy Google's ACS URL and Entity ID into Entra, and Entra's IdP metadata into the Google SSO profile
Admin locked out of Google	SSO enforced with no break-glass path	Sign in as the cloud-only Google **super-admin** excluded from SSO; keep this path always available

C. Provisioning drift or duplicates (P2/P3)

Symptom	Cause	Remediation
Objects conflict or flip-flop	Same object mastered by **both** SCIM push (P2) and GCDS pull (P3) — two sources of truth	Pick one master per object; disable the other for those objects
Users not provisioned into Google	Users not assigned/in scope of the Entra provisioning app, or SCIM not started on Google	Assign users/groups to the app; confirm the Google SCIM endpoint is enabled and provisioning is started
Leavers keep access	De-provisioning not configured — JIT used as the source of truth	Use P2/P3 for lifecycle so removal flows through; treat JIT as convenience only
Wrong OU placement	Attribute/OU mapping incomplete	Correct the provisioning attribute mappings (or GCDS mapping rules)

D. Third-party app SSO/provisioning (P5/P6)

Symptom	Cause	Remediation
App SSO works but provisioning does not	The app does not support SCIM from the chosen IdP, or the endpoint/token is wrong	**Verify the app's current capability in the vendor's docs**; use the app's supported connector/import where SCIM is unavailable (e.g. some ServiceNow configurations)
Users authenticate but have no/incorrect roles	Roles not carried in the assertion/token, or group→role mapping missing	Map groups/claims to app roles (P5 authZ); do not use per-user local admin

Symptom	Cause	Remediation
Local app accounts still in use	Snowflake — app not federated	Move the app to federated SSO (P5) and disable local passwords
SAP federation behaves unexpectedly	Brokering topology (SAP identity/BTP service → enterprise IdP) not as assumed	**Verify the exact SAP brokering topology in the vendor's docs and your tenant** before troubleshooting further

E. API / data access (P7)

Symptom	Cause	Remediation
Integration over-permissioned in the report	Scopes granted beyond what the integration uses	Cut to least privilege; prefer read-only scopes where it only reads
Long-lived shared secret flagged	Credential sprawl anti-pattern	Move to OAuth client-credentials / workload identity federation (short-lived, no stored secret); assign a named owner

F. Coverage/exception report shows unclassified integrations

- An integration that matches no catalog pattern is a **snowflake** by definition. Decide: bring it to

the matching pattern (P1–P7) via `GWINTEG-Remediation.ps1` (preview, confirm, apply), or record it as a **deliberate exception** with an owner and review date in the exception register. The report should never contain silent, unexplained snowflakes.

G. A legitimate integration broke after remediation

Symptom	Cause	Remediation
App or job fails after a grant/scope was changed	The removed permission was actually in use	Re-grant it (change is reversible); always run `GWINTEG-Remediation.ps1` in `-WhatIf` first and confirm with the app owner before applying

H. Diagnostics quick reference

```
Connect-MgGraph -Scopes 'Application.Read.All','Directory.Read.All','Policy.Read.All','Synchronization.Rea
# SSO/federation + provisioning apps (service principals with SSO mode or a sync job)
Get-MgServicePrincipal -All | Select-Object DisplayName, AppId, PreferredSingleSignOnMode
# provisioning (synchronization) jobs on an app
Get-MgServicePrincipalSynchronizationJob -ServicePrincipalId your-service-principal-id
# an app's granted application permissions and delegated grants (scope/least-privilege review)
Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId your-service-principal-id
Get-MgOauth2PermissionGrant -Filter "clientId eq 'your-service-principal-id'"
```

I. When to escalate

If an integration cannot be attributed to any owner, holds broad application permissions or a long-lived shared secret, and shows unexpected activity, treat it as a potential compromise: revoke its grants and credentials, disable the service principal (`AccountEnabled` false), and engage your incident-response process.

7. Best Practices — Enterprise Integration Patterns

- **Standardize on the catalog — one way to connect.** Every new integration picks a pattern (P1–P7), it does not invent one. Reuse is the whole point; a named pattern applied consistently beats seven clever one-offs.
- **One source of truth per object.** Master each user, group, and app account in exactly one directory and synchronize outward (SCIM push P2 **or** GCDS pull P3, never both). Two masters is the drift-and-conflict anti-pattern.
- **Federate authentication everywhere.** Prefer SAML/OIDC SSO to the enterprise IdP for every app that supports it (P1/P5). Local app passwords are snowflakes and credential-sprawl risk — design them out.
- **Drive authorization from groups and claims.** Carry roles in the assertion/token and map groups → app roles, so entitlements follow directory membership and reviews, not per-user local admin.
- **Least privilege on API and data access.** Use OAuth client-credentials / workload identity federation with scoped access (P7); never grant `*.ReadWrite.All`-style scopes "to be safe," and prefer read-only where the integration only reads.
- **Kill credential sprawl.** No shared service accounts, no long-lived secrets, no passwords in scripts. Prefer short-lived, federated credentials and give every service identity a named owner.
- **Automate the full lifecycle — including de-provisioning.** Provisioning that creates accounts but never removes leavers is worse than none; test the remove path (P2/P3/P6), not just the add path.
- **Verify third-party app capabilities in the vendor's docs and your tenant.** Whether an app supports SCIM from a given IdP, only SAML, or only a legacy connector changes over time — confirm it, don't assume it. The catalog names the pattern; the app decides what it supports.
- **Document coexistence with a cutover order.** During M365↔Google overlap, name the IdP owner per workload and the order (identity → SSO → provisioning → mail/data) under change control.
- **Stage and preview every change.** Federation, provisioning, and remediation changes can break a live integration — pilot on a scoped app set, run remediation in `-WhatIf` first, confirm with the owner, then apply to the reviewed integration only.
- **Keep a break-glass path outside SSO.** Always retain cloud-only emergency admins (Entra) and a

Google super-admin that can sign in outside federation, so an SSO change can never lock you out.

• **Make conformance recurring, not one-time.** New integrations appear constantly; re-run discovery and the coverage/exception report on a cadence so snowflakes are caught and classified as they emerge.

• **Maintain the Integration-Patterns reference doc.** Keep the catalog, the IdP-direction table, the capability matrix, and the exception register current — it is the estate's source of truth.

8. Deliverables Summary — Enterprise Integration Patterns

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	The pattern-catalog approach — the five integration dimensions, the P1–P7 reference patterns, and the anti-patterns the catalog exists to kill
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	The reference patterns: Entra↔Google federation + provisioning (P1–P3), M365 coexistence (P4), third-party apps ServiceNow/SAP/Databricks (P5/P6), and API/data access (P7) — recommended authN/authZ + data access, standards, and anti-patterns
5. Testing Guide	Pattern-conformance gates with pass/fail
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`GWINTEG-Discovery.ps1`	Inventories the Entra-side integration estate — SSO/federation apps, SCIM provisioning jobs, service principals, app-role assignments, delegated OAuth grants, and credentials — via Microsoft Graph (read-only)
`GWINTEG-Validation.ps1`	Checks pattern conformance — federation present, one source of truth for provisioning, federated app SSO, scoped API access — and reports non-conforming integrations (read-only)

Script	What it does
`GWINTEG-Report.ps1`	Integration coverage / exception report — classifies each integration against the P1–P7 catalog and lists snowflakes, over-broad scopes, and credential risk (read-only)
`GWINTEG-Remediation.ps1`	Safely brings a reviewed snowflake toward its target pattern (e.g. revokes over-broad grants); `-WhatIf` by default
`GWINTEG-Set-ScopeTargeting.ps1`	Builds and previews the pilot scope and pattern-owner groups for a staged rollout

Integration-Patterns reference doc — outline

The primary architectural deliverable. Recommended structure:

1. **Purpose & scope** — the estate covered (Entra, Google, M365 coexistence, named third-party apps).
2. **The five integration dimensions** — federation, provisioning, app SSO, API/data access, coexistence.
3. **Pattern catalog (P1–P7)** — for each: intent, authN, authZ, data access, standard, anti-pattern.
4. **IdP-direction table** — per workload/app: chosen IdP, relying party, protocol (SAML/OIDC).
5. **Provisioning map** — per object type: source of truth and mechanism (SCIM P2 / GCDS P3 / JIT).
6. **Third-party app capability matrix** — per app: SSO protocol, provisioning mechanism, role model, and "verify in tenant" flags.
1. **Coexistence cutover plan** — per workload: IdP owner and cutover order.
2. **Standards & anti-patterns** — the consistent authN/authZ/data-access rules and what to avoid.
3. **Exception register** — deliberate deviations, with owner and review date.
4. **Conformance cadence** — how discovery/report run and how new integrations are classified.

What the buyer receives

- This implementation guide and the pattern catalog as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess → Configure → Pilot → Validate → Expand.

Outcome

A consistent, standards-based integration estate — Entra↔Google federation and provisioning, M365 coexistence, and third-party apps all built to a named pattern with consistent authentication, authorization, and data access — snowflakes and credential sprawl designed out, exceptions visible and few, proven on a pilot before scaling across the estate.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWINTEG-Discovery.ps1

Purpose: Read-only inventory of the Microsoft Entra ID side of the integration estate that the pattern catalog governs. Per enterprise application (service principal) it reports the single sign-on mode (SAML / OIDC / password / none), whether a SCIM provisioning (synchronization) job exists, the application (app-role) permissions it holds, the delegated OAuth grants, the owners, and the client-secret and certificate expiry. This is the baseline you classify against patterns P1-P7. Google-side SSO profiles and third-party-app capabilities are recorded separately in the capability matrix; this script covers the Microsoft/Entra plane only. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\GWINTEG-Discovery.ps1 Optional: -CsvPath .\gwinteg-inventory.csv to export a flat CSV. Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. Use 'your-tenant-id' if prompted to select a tenant. EXPECTED RESULTS A per-application listing of SSO mode, provisioning presence, application/delegated permissions, owners, and credential expiry. No changes are made (read-only) — pass. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\GWINTEG-Discovery.ps1 Optional: -CsvPath .\gwinteg-inventory.csv to export a flat CSV. Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. Use 'your-tenant-id' if prompted to select a tenant. EXPECTED RESULTS A per-application listing of SSO mode, provisioning presence, application/delegated permissions, owners, and credential expiry. No changes are made (read-only) — pass. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

GWINTEG-Remediation.ps1

Purpose: Applies the safe, reversible remediation that moves a reviewed integration toward the catalog: it revokes the over-broad application-permission app-role assignments and risky delegated OAuth grants on a named app (the P7 least-privilege step and the first move off a snowflake). It is intended for an app you have already reviewed in GWINTEG-Report.ps1 and confirmed with the app owner. Defaults to -WhatIf, so a plain run only PREVIEWS what it would revoke and changes nothing. Re-federating an app (moving password SSO to SAML/OIDC, enabling SCIM) and any third-party-app or Google-side change are done in those consoles, not by this script. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GWINTEG-Remediation.ps1 -AppId your-app-id Apply after review: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Remediation.ps1 -AppId your-app-id -WhatIf:$false Scope switches: -RevokeDelegated / -RevokeAppRoles (default: both). Required Graph scopes: Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All. EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run GWINTEG-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES White-label — the repeatable way. Write action — preview (-WhatIf) by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GWINTEG-Remediation.ps1 -AppId your-app-id Apply after review: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Remediation.ps1 -AppId your-app-id -WhatIf:$false Scope switches: -RevokeDelegated / -RevokeAppRoles (default: both). Required Graph scopes: Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All. EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run GWINTEG-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES White-label — the repeatable way. Write action — preview (-WhatIf) by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

GWINTEG-Report.ps1

Purpose: Read-only report that classifies each Entra-side enterprise application against the pattern catalog and surfaces the exceptions. For every app it infers a likely pattern from its signals (SAML/OIDC SSO -> P1/P5 federated SSO; a SCIM provisioning job -> P2/P6 provisioning; password SSO or none -> snowflake candidate), then lists the coverage summary and three exception buckets: snowflakes (no federated pattern), over-broad-scope integrations (P7), and credential risk (no-expiry / long-lived / many secrets). Use it to build the exception register and to target GWINTEG-Remediation.ps1. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWINTEG-Report.ps1 Optional: -CsvPath .\gwinteg-coverage.csv to export the per-app classification. Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. EXPECTED RESULTS A coverage summary (how many apps map to each pattern) and exception lists (snowflakes, over-broad scopes, credential risk). No changes are made — pass. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Report.ps1 Optional: -CsvPath .\gwinteg-coverage.csv to export the per-app classification. Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. EXPECTED RESULTS A coverage summary (how many apps map to each pattern) and exception lists (snowflakes, over-broad scopes, credential risk). No changes are made — pass. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

GWINTEG-Set-ScopeTargeting.ps1

Purpose: Creates (or previews) the security groups this solution targets for a staged pattern rollout: the pilot scope "Integration Patterns Pilot Scope - Pilot" (the representative apps/users you validate the target patterns on) and the pattern-owner group "Integration Patterns Owners - Tenant - Steady" (the architects who own the catalog and review exceptions). Use the pilot scope to stage P1-P7 on a small app set before expanding across the estate. By default it PREVIEWS (no changes); pass -Create to create groups that do not yet exist. HOW TO RUN Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\GWINTEG-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Set-ScopeTargeting.ps1 -Create Required Graph scopes: Group.Read.All + Directory.Read.All (preview); Group.ReadWrite.All + Directory.Read.All (with -Create). EXPECTED RESULTS In preview: whether each named group exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create)

How to run: Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\GWINTEG-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Set-ScopeTargeting.ps1 -Create Required Graph scopes: Group.Read.All + Directory.Read.All (preview); Group.ReadWrite.All + Directory.Read.All (with -Create). EXPECTED RESULTS In preview: whether each named group exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create)

GWINTEG-Validation.ps1

Purpose: Read-only conformance check against the pattern catalog. For each enterprise application it evaluates the standards the catalog requires: federated SSO present where an app authenticates users (P1/P5), a provisioning job where lifecycle is expected (P2/P6), no over-broad application permissions on integrations (P7 least privilege), no long-lived or no-expiry credentials (credential-sprawl anti-pattern), and an assigned owner. It reports each app as CONFORMS or the specific conformance issue. Optionally focus one app with -CheckAppId. Google-side and third-party-app internals are verified in those consoles; this covers the Entra plane. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWINTEG-Validation.ps1 Focus one app: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Validation.ps1 -CheckAppId your-app-id Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. EXPECTED RESULTS A per-app pass/fail: CONFORMS, or a list of the conformance issues found (missing federation, missing provisioning, over-broad permission, risky credential, no owner). No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CheckAppId)

How to run: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Validation.ps1 Focus one app: powershell -ExecutionPolicy Bypass -File .\GWINTEG-Validation.ps1 -CheckAppId your-app-id Required Graph scopes (read-only): Application.Read.All, Directory.Read.All, Policy.Read.All, Synchronization.Read.All. EXPECTED RESULTS A per-app pass/fail: CONFORMS, or a list of the conformance issues found (missing federation, missing provisioning, over-broad permission, risky credential, no owner). No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CheckAppId)